

# PROYECTO INTERMODULAR

## Memoria Final

IES SALADILLO · ALGECIRAS — Departamento de Informática

Ciclo Formativo de Grado Medio — Sistemas Microinformáticos y Redes (2º SMR)

## Diseño, implementación y despliegue de la infraestructura de red y plataforma web para una autoescuela

**Alumnos:** Matías Álvaro Quiriquino El Ashuh · Yeray Sampalo Pérez

**Junio 2026**

## AUTOESCUELA AUTOQUIRAY — Datos del proyecto



*Auto* **Quiray**

### **Alumnos:**

- Matías Álvaro Quiriquino El Ashuh
- Yeray Sampalo Pérez

**Tutor del proyecto:** José Luis González Luna

**Curso académico:** 2025 / 2026

**Convocatoria:** Primera Ordinaria — Junio 2026

**Fecha de exposición:** Jueves 4 de junio de 2026 — 08:45 a 09:10

**Repositorio del código:** <https://github.com/Timmo230/autoquiray>

## Declaración de autoría

Los abajo firmantes declaran que el presente proyecto, titulado “*AUTOESCUELA AUTOQUIRAY — Diseño, implementación y despliegue de la infraestructura de red y plataforma web para una autoescuela*”, es fruto de su trabajo personal, que no contiene fragmentos de otros autores no citados explícitamente y que todas las fuentes consultadas se encuentran referenciadas en el apartado de bibliografía y webgrafía.

Las decisiones técnicas, el diseño de red, el código fuente alojado en el repositorio GitHub Timmo230/autoquiray y la documentación que acompaña al proyecto han sido elaboradas por los firmantes durante el curso 2025/2026, bajo la tutoría del profesor D. José Luis González Luna.

Algeciras, 4 de junio de 2026.

### Firmado:

Matías Álvaro Quiriquino El Ashuh Yeray Sampalo Pérez

## Resumen

AUTOQUIRAY es un proyecto integral que aborda la informatización de una autoescuela ficticia, abarcando tanto la **infraestructura de red** (segmentación lógica en VLANs, servidor de dominio, DHCP, almacenamiento, copias de seguridad) como la **plataforma web** que sostiene la operativa académica diaria (gestión de tests teóricos, reserva de clases prácticas, consultas entre alumnos y profesorado, paneles de control y dashboards).

El proyecto se desarrolla en Laravel 12 sobre PHP 8.2, servido por Apache 2.4 + PHP-FPM contra una base de datos MariaDB 10.11. La analítica de uso se delega a una instancia auto-hospedada de Plausible Analytics ejecutándose en Docker. La infraestructura de red se ha implementado físicamente en el aula sobre un switch de capa 3 y se ha simulado integralmente en Cisco Packet Tracer. El control de versiones se realiza mediante Git y GitHub, con dos ramas (main para entrega y web para desarrollo activo).

**Palabras clave:** Laravel, PHP, MariaDB, Apache, Docker, VLAN, Active Directory, DHCP, Plausible, Cisco Packet Tracer, Git, autoescuela.

**Centro:** IES Saladillo

**Módulo:** Sistemas Microinformáticos y Redes — 2º GM

**Profesor:** José Luis González Luna

**Fecha:** 19/05/2026 · **Versión:** 1.1 (Diseño)

# AUTOESCUELA AUTOQUIRAY

## **Sumario**

1. Resumen ejecutivo
2. Introducción
3. Objetivos
4. ¿En qué nos diferenciamos?
5. Análisis previo
6. Equipamiento previsto
7. Fase de diseño (VLANs, servicios, seguridad, backup)
8. Diseño de la aplicación web AUTOQUIRAY
9. Stack tecnológico completo
10. Aviso al profesor

## **Resumen ejecutivo**

La Autoescuela AUTOQUIRAY ha solicitado la informatización completa de sus instalaciones para mejorar la gestión académica y administrativa. El objetivo es conectar digitalmente todas las áreas: recepción, aulas, profesorado y dirección, mediante una red que permita el acceso a los servicios internos, el servidor web y los recursos compartidos.

Con esta implementación, la autoescuela podrá administrar alumnos, profesores, tests teóricos, reservas de clases prácticas y documentos desde un sistema centralizado. También se busca aumentar la seguridad de los datos y optimizar el trabajo del personal mediante una red eficiente y moderna.

Beneficios principales:

- Centralización de la información y control de accesos.
- Acceso seguro a internet y a la plataforma web.
- Simplificación de tareas administrativas.
- Mayor disponibilidad, seguridad y escalabilidad.

## **Introducción**

La digitalización de los procesos internos es una necesidad. AUTOQUIRAY realiza muchas de sus gestiones manualmente, lo que genera lentitud y errores.

El proyecto propone la creación de una infraestructura tecnológica compuesta por una red segmentada, servidores, estaciones de trabajo, impresoras en red y una plataforma web para la gestión de alumnos y tests. Todo bajo un esquema seguro, mantenible y preparado para el crecimiento futuro.

## **Objetivos**

- Implementar una red estructurada con VLANs para separar departamentos.

- Desplegar un servidor central para servicios de dominio, web y base de datos.
- Crear una plataforma web donde los alumnos puedan realizar tests y registrar su progreso.
- Configurar políticas de seguridad, copias de seguridad y accesos restringidos.
- Garantizar conectividad estable a internet y entre equipos.

## ***¿En qué nos diferenciamos?***

A diferencia de otras soluciones genéricas, AUTOQUIRAY está adaptado a las necesidades de una autoescuela en funcionamiento, no a una empresa nueva. Se prioriza facilidad de uso, gestión automatizada de usuarios e integración de todos los servicios (web, dominio, almacenamiento y backup) en un único sistema.

Los alumnos acceden mediante miniordenadores en modo quiosco que solo permiten navegar a la web de tests, garantizando seguridad y control del entorno. Profesorado y administración tienen acceso completo al servidor y recursos compartidos.

## ***Análisis previo***

La autoescuela cuenta con:

- 1 recepción (acceso a servidor e internet).
- 4 profesores (2 mañana, 2 tarde) con acceso a servidor, web e internet.
- 2 aulas con 10 alumnos cada una (acceso solo al servidor web de tests, sin internet).
- 1 jefe en oficina privada (acceso total).
- 1 sala de servidores con dos máquinas (AD y Web).

Cada grupo está en una VLAN distinta.

## ***Equipamiento previsto***

- 20 miniordenadores Windows 11 Pro.
- 7 ordenadores de oficina (profesorado, jefe, recepción).
- 3 impresoras en red.
- 1 servidor físico principal con virtualización (AD, Web, Backup).
- 1 NAS Synology con discos externos para datos y copias de seguridad.
- 2 switches gestionables de 24 puertos con link aggregation.
- 1 router/firewall profesional.
- 2 puntos de acceso Wi-Fi profesionales.

## ***Fase de diseño***

### **Tabla de VLANs**

|           |    |             |          |                |
|-----------|----|-------------|----------|----------------|
| VLAN 10 — | 14 | 1–9 y 17–20 | Switch 1 | Todos          |
| Todos     |    |             |          |                |
| VLAN 20 — | 1  | Fa 0/11     | Switch 1 | 11, 22, 23, 24 |

|                          |    |              |          |                                  |
|--------------------------|----|--------------|----------|----------------------------------|
| Dirección                |    |              |          |                                  |
| VLAN 30 — Profesores     | 6  | Fa 0/12–0/16 | Switch 1 | 22, 23, 24                       |
| VLAN 40 — Alumnos        | 20 | Fa 0/1–0/20  | Switch 2 | 1–20, 21, 23                     |
| VLAN 40 — Alumnos (ext.) | 1  | Fa 0/21      | Switch 1 | 23, 24                           |
| VLAN 50 — Admin          | 1  | Fa 0/10      | Switch 1 | 10, 22, 23, 24                   |
| VLAN 60 — Internet       | 1  | Fa 0/21      | Switch 2 | 1–20, 21, 23 y 24 (Sw 1)         |
| Serv. Dominio            | 1  | Fa 0/22      | Switch 1 | 10, 11, 22, 24                   |
| Serv. DHCP               | 1  | Fa 0/23      | Switch 1 | 10, 11, 12–16 (Sw 1 + 1–20 Sw 2) |

**VLAN 10:** predeterminada para puertos sin configurar.

**VLAN 20:** ve su propia VLAN; puerto 24 enlace trunk al otro switch para salir a internet; puerto 22 al servidor de dominio para trabajar en red con carpetas; DHCP para asignar IP.

**VLAN 30:** ve servidor de dominio (22), DHCP (23) e internet (24).

**VLAN 40:** configurada en Switch 2; añadida en Switch 1 para asignar IPs por el puerto 23 sabiendo que los equipos están en el Switch 2 por el puerto 24 (trunk) y salir a internet.

**VLAN 50:** administración. Ve su puerto (10), el servidor de dominio (22), DHCP (23) e internet (24).

**VLAN 60:** internet por puerto 21 del Switch 2; ve servidor web (23), VLAN 40 alumnos (1–20) y puerto 24 del Switch 1.

**Servidor de dominio:** se ve a sí mismo en Switch 1 (22) y a la VLAN 30 (12–16 Sw 1).

**Servidor DHCP:** ve todas las VLANs para suministrar IPs (administración 10, dirección 11, profesores 12–16, alumnos 1–20 del Sw 2).

## Servicios

- Controlador de Dominio (AD) para usuarios, grupos y políticas.
- Servidor web para la plataforma de tests y base de datos de alumnos.
- Servidor de archivos compartido para documentos administrativos y de profesorado.
- Servidor de backup con copias diarias incrementales y semanales completas.
- DHCP/DNS internos para gestión de red automatizada.

## Seguridad

- Firewall con control de acceso por VLAN.
- Políticas de contraseñas para el personal y permisos restringidos.

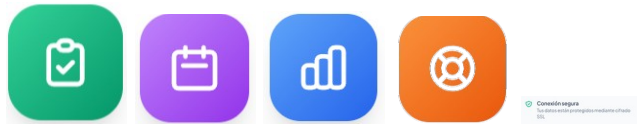
- Antivirus actualizado y copias cifradas.
- Actualización mensual de sistemas.

## Plan de backup

- Copia completa semanal e incremental diaria. Las incrementales se borran al octavo día, se conserva una completa semanal y al mes se reserva una mensual; se reinicia el ciclo el mes siguiente.
- Destino: NAS local Synology.
- Verificación mensual de restauración.

## Diseño de la aplicación web AUTOQUIRAY

Funcionalidades principales de la plataforma:



*Tests teóricos · Reserva de clases · Seguimiento de progreso · Atención al alumno · Conexión segura*

AUTOQUIRAY es la plataforma web del proyecto. Se diseña para tres roles:

- **Alumno:** ve tipos de test, hace tests teóricos, ve resultados y corrección, reserva y cancela clases, gestiona perfil/matricula/seguridad, envía consultas al profesorado.
- **Profesor:** dashboard de alumnos, responder consultas, crear tests, crear clases sobre horarios existentes.
- **Administrador:** dashboard de profesorado con métricas, detalle de actividad, crear usuarios, crear horarios.

## Arquitectura lógica

- **Autenticación:** `Auth::attempt()` + verificación de rol contra las tablas `users` → `user_is_assigned_types` → `types` mediante `RoleMiddleware`.
- **Tests teóricos:** profesor crea test → alumno responde → opciones en `student_selects_options` → resultado en `student_completes_tests`.
- **Clases prácticas:** admin crea `timetables` → profesor crea `classes` desde un horario → alumno reserva vía `students_reserves_classes`. Cancelación bloqueada si la clase ha comenzado.
- **Consultas:** alumno envía mensaje (`student_questions`) → profesor responde (`answers`).
- **Dashboards:** SQL agregado en `EmployeesPlaceService`.

## Diseño de la base de datos

- **Diseño conceptual:** entidades, relaciones y atributos.

- **Diseño lógico:** tablas con claves primarias/foráneas/únicas y relaciones en notación pata de cuervo.

## Diseño visual

Mockups previos a la codificación:

- Páginas de estudiantes.
- Páginas de profesorado.
- Páginas de administración.

## *Stack tecnológico completo*

No solo Laravel: el proyecto integra una pila amplia de herramientas. Se listan **todas** las que intervienen:

## Sistema operativo y servidor

- Ubuntu Server LTS 22.04 (servidor web).
- Windows Server (AD y DHCP).
- Windows 11 Pro (clientes).
- Apache 2.4 (HTTP).
- PHP 8.2-FPM (intérprete).
- MariaDB 10.11 / MySQL 8.0 (base de datos).

## Backend

- **Laravel 12** — framework PHP.
- **Composer** — dependencias PHP.
- **Eloquent ORM**, **Blade**, middleware RoleMiddleware.
- **Laravel Pint** — linter.
- **Laravel Pail** — visor de logs.
- **PHPUnit + Mockery + Faker** — testing.
- **Laravel Tinker** — REPL.

## Frontend

- **Bootstrap 5** — UI base.
- **SCSS** — variables de paleta y mixins (resources/scss/mains/main.scss).
- **CSS modular** por vista (resources/css/\*.css).
- **JavaScript modular** (resources/js/\*.js).
- **Vite 7** — pipeline declarado.
- **Tailwind CSS 4** + @tailwindcss/vite.
- **Axios** — cliente HTTP.

## Operaciones y herramientas

- **Git** — control de versiones.
- **GitHub** — repositorio remoto (Timmo230/autoquiray).
- **Visual Studio Code** — IDE.
- **Docker + Docker Compose** — despliegue aislado de **Plausible Analytics** para telemetría de uso.
- **Plausible Analytics** — analítica autoalojada.
- **Cisco Packet Tracer** — simulación de la red lógica.
- **VirtualBox / Hyper-V** — virtualización de servidores.
- **OpenSSH** — administración remota.
- **phpMyAdmin / DBeaver** — gestión de la base de datos.
- **Postman / curl** — pruebas de endpoints.
- **systemd** — gestión de servicios (Apache, PHP-FPM, MariaDB, worker de cola).
- **UFW** — firewall del servidor.

AUTOQUIRAY corre **directamente** sobre Apache + PHP-FPM + MariaDB. **Solo** Plausible está contenedorizado en Docker, comunicándose con la web por HTTP para recibir eventos.

## AVISO JOSE LUIS

Diseño de la parte lógica y física con mayor detalle en el archivo Proyecto\_MatiasQuriquino\_YeraySampalo.pkt adjunto a esta carpeta.

## Fase de implementación y simulación

### 1. Justificación de recursos y simulación

En este apartado se detallan los medios técnicos empleados para alcanzar el **Producto Mínimo Viable (PMV)** de AUTOQUIRAY, diferenciando entre los activos físicos disponibles en el aula, los servicios reales desplegados sobre ellos y las estrategias de simulación adoptadas para cubrir aquellas piezas de la infraestructura que no se podían reproducir físicamente.

El proyecto tiene dos capas claramente diferenciadas:

- Una **capa de red e infraestructura física** (switches, VLANs, AD, DHCP, servidor Apache, base de datos).
- Una **capa de aplicación web** (AUTOQUIRAY), desarrollada en Laravel 12, desplegada sobre Apache + MySQL/MariaDB en el servidor del aula y monitorizada con Plausible Analytics ejecutándose en Docker.

### 1.1 Recursos reales (hardware y software físico)

Elementos con los que se ha trabajado directamente:



- **Infraestructura de red:** Switch de Capa 3 real, fundamental para configurar el enrutamiento inter-VLAN y la segmentación lógica del aula.
- **Servidores y servicios:** hardware físico para el despliegue de:
  - **Servidor Web + Base de datos:** máquina Linux con Apache 2.4, PHP 8.2-FPM y MariaDB 10.11 sirviendo la aplicación AUTOQUIRAY.
  - **Servidor de infraestructura:** Active Directory (AD) y DHCP sobre Windows Server.
- **Entorno de desarrollo:**
  - **IDE:** Visual Studio Code con extensiones de PHP Intelephense, Laravel Blade, GitLens y Tailwind CSS IntelliSense.
  - **Backend:** Laravel 12 (PHP 8.2+) con Eloquent, middleware de roles, Blade, queue worker y Pint para lint.
  - **Frontend:** Bootstrap 5 cargado desde public/node\_modules, SCSS propio compilado a CSS, JS modular y Vite + Tailwind 4 declarados como pipeline moderno.
  - **Control de versiones:** Git + repositorio en GitHub (Timmo230/autoquiray).
- **Terminales:** ordenadores configurados como clientes para pruebas de conectividad, acceso al dominio y navegación a la web de tests.

## 1.2 Recursos simulados y estrategias de sustitución

El proyecto original planteaba una inversión de hardware superior a la disponible. Se han aplicado las siguientes sustituciones:

- **Virtualización de servidores:** se preveían 4 servidores físicos independientes. Se ha consolidado mediante virtualización (VirtualBox / Hyper-V) los roles de Web, AD y DHCP en dos máquinas físicas.
- **Simulación de almacenamiento (NAS):** el NAS Synology previsto se ha sustituido por carpetas compartidas y permisos en Active Directory.
- **Simulación de dispositivos periféricos:** no se disponía de las 3 impresoras de red ni de los 28 equipos finales. Se ha simulado el comportamiento con máquinas virtuales y los equipos del aula, validando que DHCP y GPOs funcionarían en un despliegue masivo.
- **Maquetación lógica completa con Cisco Packet Tracer:** ver archivo Proyecto\_MatiasQuriquino\_YeraySampalo.pkt adjunto. Reproduce los segmentos de red que no podían cablearse físicamente.

## 2. Stack tecnológico utilizado

A continuación se listan **todas** las tecnologías que intervienen en el proyecto, no únicamente Laravel:

### 2.1 Sistema y servidor

SO servidor

Ubuntu Server LTS / Windows 22.04 LTS para Apache;

|                |                                 |                                                                 |
|----------------|---------------------------------|-----------------------------------------------------------------|
|                | Server (mixto)                  | Windows Server para AD/DHCP                                     |
| Servidor HTTP  | Apache                          | 2.4 con módulos rewrite, headers, ssl, proxy_fcgi               |
| Intérprete     | PHP-FPM                         | 8.2 con extensiones mbstring, xml, bcmath, curl, mysql, gd, zip |
| Base de datos  | MariaDB / MySQL                 | 10.11 / 8.0                                                     |
| Cache / sesión | Driver database en MariaDB      | configurado en .env                                             |
| Cola           | Laravel Queue (driver database) | worker php artisan queue:listen                                 |

## 2.2 Backend (PHP)

- **Laravel 12** — framework principal.
- **Composer** — gestor de dependencias PHP.
- **Eloquent ORM** — modelado del dominio.
- **Blade** — motor de plantillas.
- **Middleware personalizado** RoleMiddleware — autorización por rol.
- **Laravel Pint** — linter PSR-12.
- **Laravel Pail** — visor de logs en desarrollo.
- **PHPUnit + Mockery + Faker** — testing.

## 2.3 Frontend

- **Bootstrap 5** — base de componentes y rejilla.
- **SCSS** — variables de paleta y mixins (resources/scss/mains/main.scss) compilado a main.min.css.
- **CSS modulares por vista** — resources/css/ (login.css, tests.css, dashboardAdmins.css, etc.).
- **JavaScript propio** — resources/js/ (modular: nav, test, classes, dashboards...).
- **Vite 7** — pipeline declarado para futura migración completa.
- **Tailwind CSS 4** + @tailwindcss/vite — declarado.
- **Axios** — cliente HTTP en frontend.

## 2.4 Herramientas de desarrollo y operaciones

- **Git** — control de versiones.
- **GitHub** — repositorio remoto, gestión de ramas (main, web) y revisión de cambios.
- **Visual Studio Code** — IDE.
- **Docker + Docker Compose** — utilizado para desplegar **Plausible Analytics** de forma aislada del servidor web. Plausible expone su panel y recibe eventos del frontend de AUTOQUIRAY a través de data-plausible-event.
- **Plausible Analytics** — analítica de uso, autoalojada en contenedor.
- **Cisco Packet Tracer** — simulación de la red lógica (VLANs, trunks, switches, servidores).

- **VirtualBox / Hyper-V** — virtualización de servidores.
- **OpenSSH** — administración remota del servidor.
- **phpMyAdmin / DBeaver** — administración de la base de datos.
- **Postman / curl** — pruebas manuales de endpoints.

Nota: AUTOQUIRAY **no** está contenedorizado. La aplicación corre directamente sobre Apache + PHP-FPM + MariaDB en el servidor físico del aula. Lo que sí corre en Docker es **Plausible**, que actúa como sistema externo de telemetría y se comunica con la web por HTTP.

### ***3. Proceso de instalación y configuración del servidor***

Esta sección documenta paso a paso cómo se levanta el entorno real de AUTOQUIRAY desde un Ubuntu Server limpio.

#### **3.1 Preparación del sistema**

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y software-properties-common curl unzip git ufw
sudo ufw allow OpenSSH
sudo ufw allow "Apache Full"
sudo ufw enable
```

#### **3.2 Instalación de Apache**

```
sudo apt install -y apache2
sudo a2enmod rewrite headers ssl proxy_fcgi setenvif
sudo systemctl enable --now apache2
```

Verificación: `http://<ip-servidor>/` debe mostrar la página por defecto.

#### **3.3 Instalación de PHP 8.2 y extensiones**

```
sudo add-apt-repository ppa:ondrej/php -y
sudo apt update
sudo apt install -y php8.2 php8.2-fpm php8.2-cli \
php8.2-mbstring php8.2-xml php8.2-bcmath php8.2-curl \
php8.2-mysql php8.2-gd php8.2-zip php8.2-intl
```

```
sudo a2enconf php8.2-fpm
sudo systemctl restart apache2
```

### 3.4 Instalación de MariaDB

```
sudo apt install -y mariadb-server
sudo mysql_secure_installation
```

Crear base de datos y usuario:

```
CREATE DATABASE autoquiray CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

CREATE USER 'autoquiray'@'localhost' IDENTIFIED BY 'PASSWORD_SEGURA';

GRANT ALL PRIVILEGES ON autoquiray.* TO 'autoquiray'@'localhost';

FLUSH PRIVILEGES;
```

### 3.5 Instalación de Composer y Node.js

```
# Composer
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer

# Node.js LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

### 3.6 Despliegue de AUTOQUIRAY

```
cd /var/www
sudo git clone https://github.com/Timmo230/autoquiray.git
sudo chown -R www-data:www-data autoquiray
cd autoquiray
sudo -u www-data composer install --no-dev --optimize-autoloader
sudo -u www-data cp .env.example .env
sudo -u www-data php artisan key:generate
```

Editar .env:

APP\_NAME=AUTOQUIRAY

APP\_ENV=production

APP\_DEBUG=false

APP\_URL=http://autoquiray.local

DB\_CONNECTION=mysql

DB\_HOST=127.0.0.1

DB\_PORT=3306

DB\_DATABASE=autoquiray

DB\_USERNAME=autoquiray

DB\_PASSWORD=PASSWORD\_SEGURA

Migraciones, seeders y build de assets:

sudo -u www-data php artisan migrate --force

sudo -u www-data php artisan db:seed --force

sudo -u www-data npm install

sudo -u www-data npm run build

sudo -u www-data php artisan storage:link

sudo -u www-data php artisan config:cache route:cache view:cache

### **3.7 VirtualHost de Apache**

/etc/apache2/sites-available/autoquiray.conf:

<VirtualHost \*:80>

ServerName autoquiray.local

DocumentRoot /var/www/autoquiray/public

<Directory /var/www/autoquiray/public>

AllowOverride All

Require all granted

</Directory>

```

<FilesMatch \.php$>
SetHandler "proxy:unix:/run/php/php8.2-fpm.sock|fcgi://localhost"
</FilesMatch>
ErrorLog ${APACHE_LOG_DIR}/autoquiray_error.log
CustomLog ${APACHE_LOG_DIR}/autoquiray_access.log combined
</VirtualHost>

sudo a2ensite autoquiray.conf
sudo a2dissite 000-default.conf
sudo systemctl reload apache2

```

### 3.8 Worker de cola (systemd)

```

/etc/systemd/system/autoquiray-queue.service:
[Unit]
Description=AUTOQUIRAY queue worker
After=network.target mariadb.service
[Service]
User=www-data
Group=www-data
Restart=always
WorkingDirectory=/var/www/autoquiray
ExecStart=/usr/bin/php artisan queue:work --tries=3 --timeout=90
[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable --now autoquiray-queue

```

### ***Implementación de red y dominio (Active Directory)***

Esta parte cubre el despliegue del dominio sobre Windows Server: alta de usuarios y grupos, unión de equipos cliente al dominio y publicación de los recursos compartidos. Es la capa que da

acceso controlado al personal de la autoescuela (profesorado, recepción y dirección), complementaria al servidor web Linux.

## **Usuarios, grupos y plantillas en Active Directory**

El primer paso fue crear los usuarios y los grupos en Active Directory, para después unirlos al dominio. Se definieron **plantillas de usuario** que agilizan el alta masiva: al ampliar la plantilla se hereda la configuración base y solo hay que ajustar los datos individuales.

Cada grupo va asociado al departamento correspondiente (profesores, recepción, administración) y recibe sus permisos según ese departamento. Así el control de acceso a los recursos compartidos queda gobernado por la pertenencia a grupo, no por usuario individual.

WinServer [Corriendo] - Oracle VirtualBox

Archivo
Máquina
Ver
Entrada
Dispositivos
Ayuda

Usuarios y equipos de Active Directory

Archivo
Acción
Ver
Ayuda

Usuarios y equipos de Active Dir

> Consultas guardadas

v autoquiray.local

> Built-in

> Computers

> Domain Controllers

> ForeignSecurityPrincipal:

> Managed Service Account

Users

| Nombre            | Tipo             | Descripción                  |
|-------------------|------------------|------------------------------|
| Alumnado          | Grupo de segu... |                              |
| Alumno1           | Usuario          |                              |
| Alumno10          | Usuario          |                              |
| Alumno2           | Usuario          |                              |
| Alumno3           | Usuario          |                              |
| Alumno4           | Usuario          |                              |
| Alumno5           | Usuario          |                              |
| Alumno6           | Usuario          |                              |
| Alumno7           | Usuario          |                              |
| Alumno8           | Usuario          |                              |
| Alumno9           | Usuario          |                              |
| Controlador...    | Grupo de segu... | Todos los controladores ...  |
| Controlador...    | Grupo de segu... | Se pueden clonar los mi...   |
| Controlador...    | Grupo de segu... | Los miembros de este gr...   |
| DnsAdmins         | Grupo de segu... | Grupo de administrador...    |
| DnsUpdateP...     | Grupo de segu... | Cientes DNS que tienen...    |
| Enterprise D...   | Grupo de segu... | Los miembros de este gr...   |
| Equipos del ...   | Grupo de segu... | Todas los servidores y es... |
| Grupo de re...    | Grupo de segu... | Los miembros de este gr...   |
| Grupo de re...    | Grupo de segu... | Los miembros de este gr...   |
| Invitado          | Usuario          | Cuenta integrada para el...  |
| Invitados del...  | Grupo de segu... | Todos los invitados del ...  |
| Jefe              | Usuario          |                              |
| Plantilla Alu...  | Usuario          |                              |
| Plantilla Prof... | Usuario          |                              |
| Profesor1         | Usuario          |                              |
| Profesor2         | Usuario          |                              |
| Profesor3         | Usuario          |                              |
| Profesor4         | Usuario          |                              |
| Profesorado       | Grupo de segu... |                              |
| Propietarios ...  | Grupo de segu... | Los miembros de este gr...   |
| Protected Us...   | Grupo de segu... | Los miembros de este ar...   |

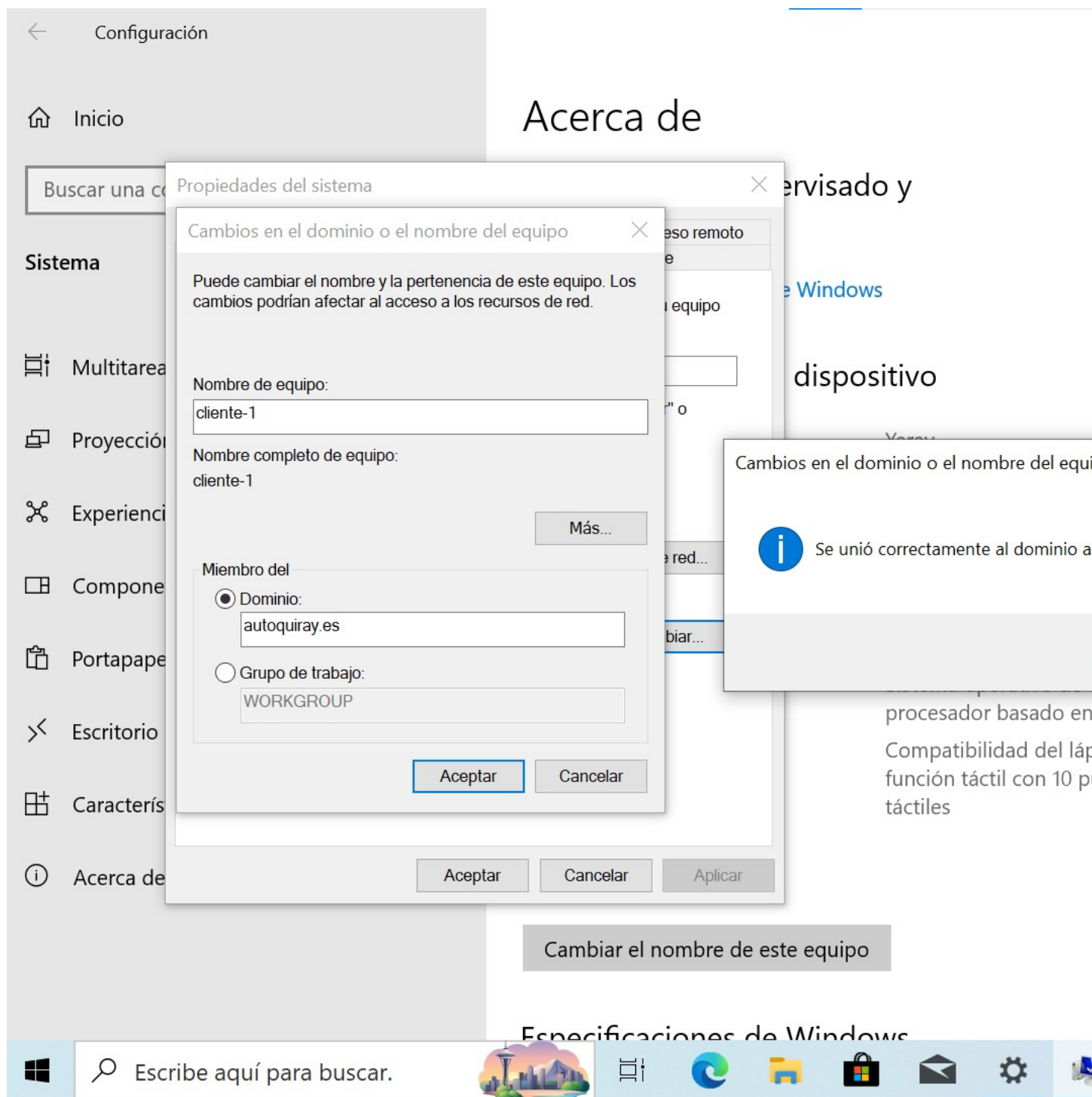
16



## **Unión de un equipo cliente al dominio**

Con los usuarios y grupos creados y los permisos asignados, se une cada equipo cliente al servidor de dominio:

1. Abrir la configuración de red del cliente y fijar como **puerta de enlace** la IP del servidor: 192.168.1.37.
2. Ir a *Cambiar el nombre de este equipo (avanzado)*.
3. Pulsar *Cambiar*, seleccionar *Dominio* e introducir el nombre del dominio al que unirse.
4. Introducir las credenciales del servidor.
5. Reiniciar. Tras el reinicio el equipo queda unido al dominio.



*Configuración de red y unión al dominio del equipo cliente*

## **Acceso a los recursos compartidos**

Una vez el cliente forma parte del dominio, se conecta a la red de recursos compartidos del servidor:

1. En el equipo cliente abrir el explorador de archivos.
2. Entrar en *Red* y activar el acceso a recursos compartidos.
3. En la barra de direcciones escribir la ruta UNC del servidor: \\192.168.1.37.
4. Introducir las credenciales de red.
5. Se muestran todas las carpetas almacenadas en el servidor.



WinServer [Corriendo] - Oracle VirtualBox

Archivo

Máquina

Ver

Entrada

Dispositivos

Ayuda



Papelera de  
reciclaje



Google Chrome



Administrador del servidor



« Servidor local »



Administrar



Panel



Servidor local



Todos los servidores



AD DS



DHCP



DNS



Servicios de archivos y...



## PROPIEDADES

Para WIN-4SR47HHNKH7

Nombre de equipo WIN-4SR47HHNKH7  
Dominio autoquiray.local

Firewall de Windows Defender Público: Desactivado

Administración remota Habilitado

Escritorio remoto Deshabilitado

Formación de equipos de NIC Deshabilitado

Ethernet 192.168.0.37, IPv6

Ethernet 2 192.168.1.37, IPv6

Versión del sistema operativo Microsoft Windows Server 2012 R2

Información de hardware innotek GmbH V

Admini

Admini

Admini

Centra

Confir

Copia

Desfr

DHCP

Diagn

Direc

DNS

Dom

Edito

Edito

Infor

Inicia

Libe

Mód

Moni

Moni

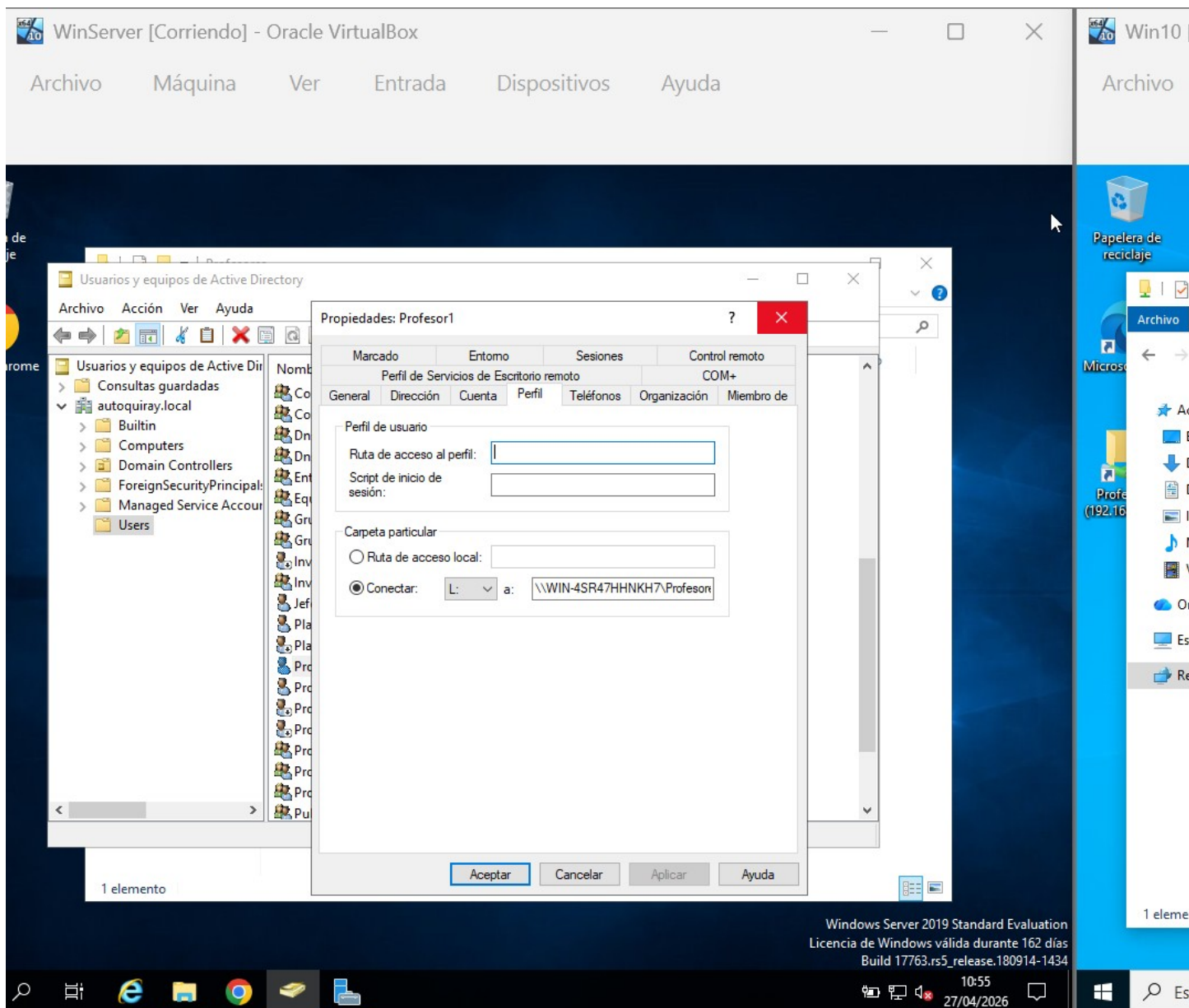
ODBC

## **Carpetas particulares por usuario**

Para que cada usuario disponga de su propio espacio de almacenamiento se configuran **carpetas particulares** automáticas:

1. En el servidor, crear la carpeta contenedora del departamento (por ejemplo profesores).
2. Entrar en el perfil del usuario en Active Directory y, en el apartado *Carpeta particular*, escribir la ruta: \\servidor\carpeta\_contenedora\carpeta\_compartida.
3. Al aplicar la configuración, dentro de la carpeta del departamento se genera automáticamente una **subcarpeta individual por cada usuario** al que se le asigne.

Cada usuario solo accede a su carpeta, garantizando aislamiento del contenido.



*Carpetas particulares generadas por usuario en el servidor*

## ***Puesta en producción con HTTPS***

El servidor sirve AUTOQUIRAY cifrado por **HTTPS**, accesible desde cualquier equipo de la red local en `https://192.168.100.248`. Como el despliegue es interno y se accede por dirección IP (no por un dominio público), no es posible emitir un certificado de una autoridad pública como Let's Encrypt (estas solo firman dominios). Por ello se ha creado una **autoridad de certificación (CA) propia** que firma el certificado del servidor.

## Esquema de confianza

1. **CA raíz local** (AutoQuiray Root CA): clave privada `ca.key` y certificado `ca.crt` con `CA:TRUE`. Es la entidad en la que confían los clientes.
2. **Certificado de servidor** (`autoquiray.crt`): firmado por la CA, con la IP del servidor en el campo *Subject Alternative Name* (IP:192.168.100.248). Es el que presenta Apache.
3. Cada equipo de la red importa **una sola vez** el `ca.crt` como autoridad de confianza; a partir de ahí el navegador muestra el candado sin avisos.

## Generación de los certificados

```
# 1. CA raíz (válida 5 años)
openssl genrsa -out ca.key 2048
openssl req -x509 -new -nodes -key ca.key -sha256 -days 1825 -out ca.crt \
  -subj "/O=AutoQuiray Local CA/CN=AutoQuiray Root CA"

# 2. Clave + petición del servidor
openssl genrsa -out autoquiray.key 2048
openssl req -new -key autoquiray.key -out server.csr \
  -subj "/O=AutoQuiray/CN=192.168.100.248"

# 3. Firma con la CA, añadiendo la IP como SAN
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out autoquiray.crt -days 825 -sha256 \
  -extfile <(printf "subjectAltName=IP:192.168.100.248, IP:127.0.0.1\
nextendedKeyUsage=serverAuth")
```

## Configuración de Apache

Se habilita el módulo SSL y se publica un *VirtualHost* en el puerto 443 que apunta al certificado firmado. Un segundo *VirtualHost* en el puerto 80 redirige todo el tráfico HTTP a HTTPS (código 301).

```
<VirtualHost *:80>
    DocumentRoot /var/www/autoquiray/public
    RewriteEngine On
    RewriteCond %{HTTPS} !=on
    RewriteRule ^/?(.*) https://%{HTTP_HOST}/$1 [R=301,L]
</VirtualHost>

<VirtualHost *:443>
    DocumentRoot /var/www/autoquiray/public
    SSLEngine on
    SSLCertificateFile    /var/www/autoquiray/deploy/ssl/autoquiray.crt
    SSLCertificateKeyFile /var/www/autoquiray/deploy/ssl/autoquiray.key
</VirtualHost>
sudo a2enmod ssl rewrite headers
sudo a2ensite autoquiray-ssl
sudo apache2ctl configtest && sudo systemctl reload apache2
```

## Soporte en Laravel

Para que las URLs generadas (`asset()`, `route()`, `url()`) usen el esquema correcto sin provocar *mixed content*, se confía en las cabeceras del proxy (`trustProxies('*)` en `bootstrap/app.php`) y se fuerza el esquema HTTPS solo cuando la petición ya es segura, en `AppServiceProvider`:

```
public function boot(Request $request): void
{
    if ($request->isSecure()) {
        URL::forceScheme('https');
    }
}
```

Así, al servir por HTTPS, todos los enlaces y recursos (CSS, JS, imágenes) se emiten en `https://`, evitando que el navegador bloquee contenido mixto.

## Importación del certificado en los clientes

- **Chrome / Chromium (Linux):** `certutil -d sql:$HOME/.pki/nssdb -A -t "C,,," -n "AutoQuiray Root CA" -i ca.crt` (Chrome usa su propio almacén NSS, independiente del sistema).
- **Windows:** instalar `ca.crt` en *Entidades de certificación raíz de confianza*.
- **Firefox / Android:** importar `ca.crt` como autoridad de confianza.

## 4. Plausible Analytics en Docker

Plausible es el único componente contenedorizado. Se despliega aparte del servidor web, en su propia red Docker.

### 4.1 Instalación de Docker

```
sudo apt install -y ca-certificates curl gnupg
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
```

```
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
```

```
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```



```
sudo usermod -aG docker $USER
```

## 4.2 Compose de Plausible (resumen)

```
/opt/plausible/docker-compose.yml (resumen):
```

```
services:
```

```
  plausible_db:
```

```
    image: postgres:16-alpine
```

```
    volumes: [ "db-data:/var/lib/postgresql/data" ]
```

```
    environment:
```

```
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

```
  plausible_events_db:
```

```
    image: clickhouse/clickhouse-server:24.3-alpine
```

```
    volumes: [ "event-data:/var/lib/clickhouse" ]
```

```
  plausible:
```

```
    image: ghcr.io/plausible/community-edition:v2.1
```

```
    depends_on: [plausible_db, plausible_events_db]
```

```
    ports: [ "8000:8000" ]
```

```
    environment:
```

```
      BASE_URL: http://plausible.local:8000
```

```
      SECRET_KEY_BASE: ${SECRET_KEY_BASE}
```

```
    volumes:
```

```
      db-data:
```

```
      event-data:
```

```
      cd /opt/plausible
```

```
      docker compose up -d
```

## 4.3 Integración con AUTOQUIRAY

En `resources/views/partials/scripts.blade.php` la web inyecta el script de Plausible apuntando al contenedor:

```
<script defer data-domain="autoquiray.local"
src="http://plausible.local:8000/js/script.js"></script>
```

Los eventos personalizados se disparan con `window.trackEvent` y los atributos `data-plausible-event` / `data-plausible-submit` ya presentes en las vistas.

## 5. Flujo de trabajo con Git y GitHub

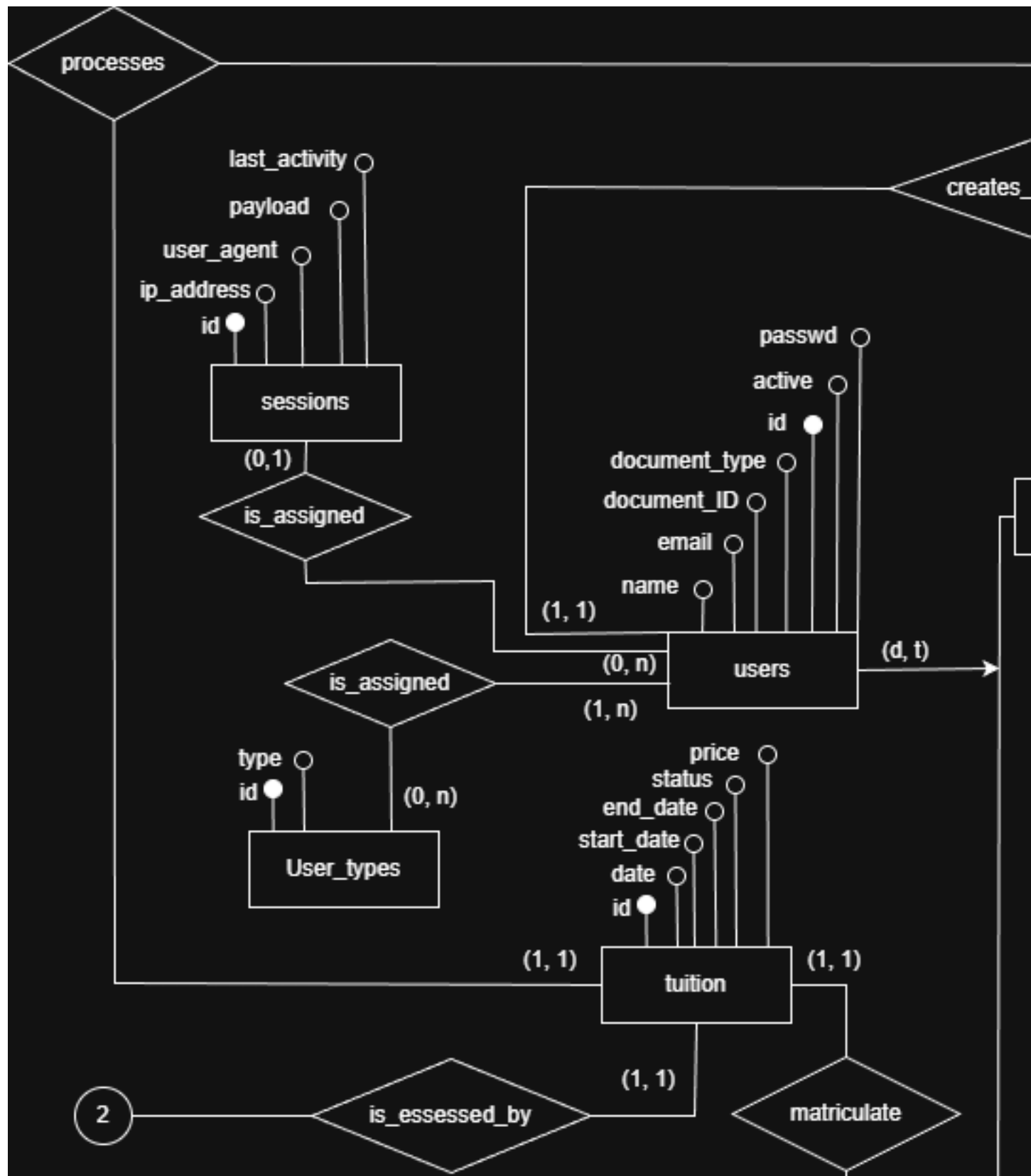
- Repositorio: <https://github.com/Timmo230/autoquiray>
- Ramas principales:
  - `main` — entrega final / producción.
  - `web` — desarrollo activo de la aplicación web.
- Commits convencionales (`feat:`, `fix:`, `chore:`, `docs:`).
- Cada despliegue en el servidor del aula se hace con:

```
cd /var/www/autoquiray
sudo -u www-data git pull origin web
sudo -u www-data composer install --no-dev --optimize-autoloader
sudo -u www-data php artisan migrate --force
sudo -u www-data npm install && sudo -u www-data npm run build
sudo -u www-data php artisan config:cache route:cache view:cache
sudo systemctl restart autoquiray-queue
```

## 6. Evidencias de la implementación

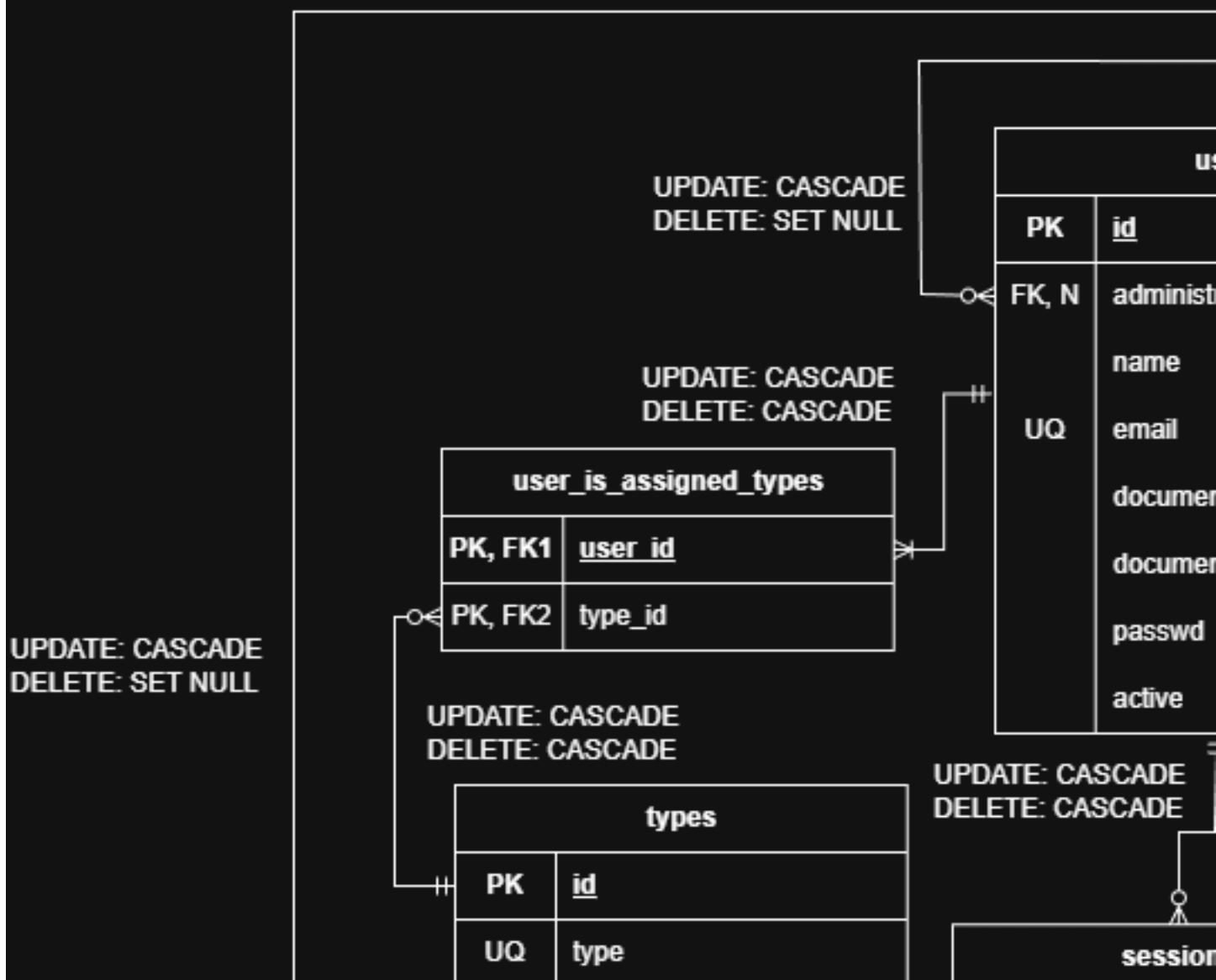
### 6.1 Diseño de la base de datos

**Diseño conceptual** — entidades, relaciones y datos guardados:



Diseño conceptual de la base de datos

**Diseño lógico** — tablas, claves primarias/foráneas/únicas y relaciones en notación pata de cuervo:



Diseño lógico de la base de datos

## 6.2 Página web

Código completo en el repositorio GitHub: <https://github.com/Timmo230/autoquiray/tree/web>

**Migración** — Laravel crea las tablas de la BD; un archivo por tabla:

```
2026_01_25_235105_create_users_table.php X
database > migrations > 2026_01_25_235105_create_users_table.php > ...
1  k?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('users', function (Blueprint $table) {
15             $table->string('id', 100)->primary();
16             $table->string('document_id');
17             $table->enum('document_type', ['DNI', 'passport']);
18             $table->string('name');
19             $table->string('email')->unique();
20             $table->boolean('active')->default(true);
21             $table->string('password');
22             $table->timestamps();
23         });
24     }
25
26     /**
27      * Reverse the migrations.
28      */
29     public function down(): void
30     {
31         Schema::dropIfExists('users');
32     }
33 };
34
```

Ejemplo de migración Laravel

**Modelo Eloquent** — representación de una tabla:

User.php X

app > Models > User.php > ...

```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Foundation\Auth\User as Authenticatable;
8  use Illuminate\Notifications\Notifiable;
9  use Laravel\Sanctum\HasApiTokens;
10
11  class User extends Authenticatable
12  {
13      use HasFactory, Notifiable;
14
15      public $incrementing = false;
16      protected $keyType = 'string';
17
18      protected $fillable = [
19          'document_id',
20          'document_type',
21          'name',
22          'email',
23          'active',
24          'administrator_id',
25      ];
26
27      protected $hidden = [
28          'password',
29      ];
30
31      public function student()
32      {
33          return $this->hasOne(Student::class, 'user_id');
34      }
35
36      public function employees()
37      {
38          return $this->hasOne(Employees::class, 'user_id');
39      }
40
```



Modelo de una tabla

**Factory** — generador de datos falsos para la BD:

```
use App\Models\Teacher;
use App\Models\Option;
use App\Models\Test;
use Illuminate\Database\Eloquent\Factories\Factory;

class QuestionTestFactory extends Factory
{
    protected $model = QuestionTest::class;

    protected static $tests = null;
    protected static $counter = 0;
    protected static $index = 0;
    public function definition(): array
    {
        if (is_null(self::$tests)) {
            self::$tests = Test::orderBy('id', 'asc')->get(['id', 'max_note'])->toArray();
        }

        $currentTest = self::$tests[self::$index];
        $maxQuestions = $currentTest['max_note'];

        if (self::$counter >= $maxQuestions) {
            self::$index++;
            self::$counter = 0;

            $currentTest = self::$tests[self::$index];
        }

        $testId = $currentTest['id'];
        self::$counter++;

        return [
            'id' => fake()->bothify('????????-????????-????????-????????'),
            'test_id' => $testId,
            'teacher_id' => Teacher::inRandomOrder()->first()->employees_id,
            'title' => rtrim($this->faker->sentence(3), '.'),
            'correct_option_id' => null,
        ];
    }
}
```

Factory de Laravel

**Seeder** — llama a los factories indicando la cantidad de registros:

```
5 use Illuminate\Database\Console\Seeds\WithoutModelEvents;
6 use Illuminate\Database\Seeder;
7 use App\Models\Student;
8 use App\Models\Exam;
9 use App\Models\Registers;

10
11 class registersSeeder extends Seeder
12 {
13     /**
14      * Run the database seeds.
15      */
16     public function run(): void
17     {
18         $created = 0;
19
20         while ($created < 1000) {
21             $student = Student::inRandomOrder()->first();
22             $exam     = Exam::inRandomOrder()->first();
23
24             $register = Registers::firstOrCreate(
25                 [
26                     'student_id' => $student->user_id,
27                     'exam_id'    => $exam->getAttributes()['id'],
28                 ],
29                 [
30                     'note' => rand(25, 30),
31                 ]
32             );
33
34             if ($register->wasRecentlyCreated) {
35                 $created++;
36             }
37         }
38     }
39 }
```

Seeder principal

Variante de seeder escrita de otra forma:

```
public function run(): void
{
    \App\Models\Permission::factory(10)->create();
}
```

Seeder alternativo

**Controller** — solicitudes a la BD para guardar o leer datos y mostrarlos en la web:

```
public function get(Request $request){
    $testId = $request->query('id');
    $userId = Auth::id();

    $row = DB::table('student_completes_tests')
        ->where('student_id', '=', $userId)
        ->where('test_id', '=', $testId)
        ->first();

    if (!$row) return redirect('/home')->with('error',

    $testMaxNote = DB::table('tests')
        ->where('id', '=', $testId)
        ->first();

    $successes = $row->last_note;
    $failed = $testMaxNote - $successes;
    $time = $row->time;

    return view('student.result', [
        'successes' => $successes,
        'time' => $time,
        'failed' => $failed,
        'max_note' => $testMaxNote,
    ]);
}
```

Controller con acceso a BD

routes/web.php — autorización por rol y rutas:

```

use App\Http\Controllers\HomeController;
use App\Http\Controllers\LoginController;
use App\Http\Controllers\hacerTestController;
use App\Http\Controllers\haciendoTestController;
use App\Http\Controllers\ResultsController;
use Illuminate\Support\Facades\Route;
use Illuminate\Support\Facades\Auth;

// Rutas publicas
Route::get('/', [HomeController::class, 'index'])->name('home');

// Rutas de autenticacion
Route::get('/login', [LoginController::class, 'showLoginForm'])->name('login');
Route::post('/login', [LoginController::class, 'login']);
Route::post('/logout', [LoginController::class, 'logout'])->name('logout');

Route::middleware(['role:student'])->group(function () {
    Route::get('/tipos_de_test', fn() => view('student.testTypes'))->name('student.testTypes');
    Route::get('/classes', fn() => view('student.classes'))->name('student.classes');
    Route::get('/contacto', fn() => view('student.contacto'))->name('student.contacto');
    Route::get('/hacer_tests', [hacerTestController::class, 'showTests'])->name('student.hacer_tests');
    Route::get('/haciendo_test', [haciendoTestController::class, 'showquestions'])->name('student.haciendo_test');
    Route::get('/resultados', [ResultsController::class, 'get'])->name('student.resultados');
    Route::post('/resultados', [ResultsController::class, 'upload'])->name('student.resultados');
});

Route::middleware(['role:teacher'])->group(function() {
    Route::get('/dashboard', fn() => view('teacher.dashboard'))->name('teacher.dashboard');
});

Route::middleware(['role:administrator'])->group(function() {
    Route::get('/panel', fn() => view('admin.dashboard'))->name('admin.dashboard');
});

```

routes/web.php

.env — datos de la base de datos, entorno (desarrollo / producción), URL, etc.:

```
APP_LOCALE=en
APP_FALLBACK_LOCALE=en
APP_FAKER_LOCALE=en_US

APP_MAINTENANCE_DRIVER=file
# APP_MAINTENANCE_STORE=database

# PHP_CLI_SERVER_WORKERS=4

BCRYPT_ROUNDS=12

LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug
|
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=autoquiray
DB_USERNAME=root
DB_PASSWORD=123

SESSION_DRIVER=database
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null

BROADCAST_CONNECTION=log
FILESYSTEM_DISK=local
QUEUE_CONNECTION=database
```

Fichero .env

**JavaScript de frontend** — una de las funciones JS:

```

finishTest: async function(qId){
  const respondidas = Object.keys(quizApp.userAnswers).length;

  if (respondidas < quizApp.totalSteps) {
    if (!confirm(`Solo has respondido ${respondidas} de ${quizApp.totalSteps}. ¿Quieres continuar?`)) {
      return;
    }
  }

  const post = await fetch('/autoquiray/resultados', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').content
    },
    body: JSON.stringify({
      responds: quizApp.userAnswers,
      testId: this.testId
    })
  });

  console.log("Enviando respuestas:", quizApp.userAnswers);
  if(post.ok){
    window.location.href = `/autoquiray/resultados?id="${this.testId}"`;
  }
  else alert('Error al guardar el test');
}

```

Función JS de frontend

SCSS — variables de paleta de colores en resources/scss/mains/main.scss:

```

sources / scss / variables.scss / $navbar
1  $navbar: #3e4454;
2  $green-btn: #30d380;
3  $green: #34d399;
4  $main: #f8fafc;
5  $footer: #1e293b;
6  $yellow: #ebb40b;
7  $grey: #858b95;
8  $dark-green: #068057;
9
10 $autoquiray-colors: (
11     "navbar": $navbar,
12     "green-btn": $green-btn,
13     "main": $main,
14     "purple": $purple,
15     "blue": $blue,
16     "orange": $orange,
17     "footer": $footer,
18     "green": $green,
19     "yellow": $yellow,
20     "grey": $grey,
21     "dark-green": $dark-green,
22 );
23
24 $theme-colors: map-merge($autoquiray-colors, $theme-colors);

```

SCSS con paleta de colores

.blade.php — combina HTML, Bootstrap y PHP:

```

<div class="row row-cols-1 row-cols-md-2 p-1 container-fluid">
    @foreach ($tests as $test)
        @php
            $registroCompletado = $testsOfStudent->firstWhere('id', $test->id);
            $clase = 'bg-white text-dark';

            if ($registroCompletado) {
                $nota = $registroCompletado->last_note;

                $clase = match (true) {
                    $nota == 30 => 'bg-success-subtle',
                    $nota >= 27 => 'bg-warning-subtle',
                    default      => 'bg-danger-subtle',
                };
            }
        @endphp
        <div class="col my-2 {{ $iconos[$categoria] }} px-2" id="{{ $test->id }}">
            <div class="card border-0 rounded-4 p-4 test-card shadow {{ $clase }}">
                <div class="d-flex justify-content-between align-items-start">
                    <div class="d-flex align-items-center">
                        
                    </div>
                    <span class="badge rounded-pill bg-success-subtle text-success px-3 py-1">
                        {{ $test->max_note }} preguntas
                    </span>
                </div>
            </div>
        </div>
    </div>

```

Vista Blade

## 7. Gestión de incidencias y cambios

Durante la implementación se produjeron las siguientes desviaciones respecto al plan inicial:

- **Reorientación parcial a Windows Server:** la primera versión del servidor web se planificó completamente sobre Ubuntu. Por restricciones de cronograma y por priorizar la usabilidad del personal de la autoescuela (familiarizado con Windows), AD y DHCP se trasladaron a Windows Server. La web sigue corriendo sobre Apache + MariaDB en Linux.
- **Virtualización en lugar de hardware físico:** la demora en la recepción de materiales hizo inviable montar 4 servidores físicos. Se virtualizaron sobre dos hosts.
- **Diseño de red en Cisco Packet Tracer:** ante la falta de hardware completo, la topología



se validó en .pkt, manteniendo el diseño lógico idéntico al planificado.

- **Plausible aislado en Docker:** decisión técnica para no acoplar la analítica al servidor web y poder destruir/recrear el contenedor sin afectar a AUTOQUIRAY.

## 8. Manual de usuario / ficha técnica

### 8.1 Gestión de conectividad (DHCP)

- **Obtención de IP:** asignación dinámica. Lease time fijado en **7 días**.
- **Reservas de IP:** equipos críticos con IP estática. Ejemplo: el equipo del Profesor 1 está vinculado permanentemente a **192.168.1.51**.

### 8.2 Acceso al servidor web

- **Servicios necesarios en ejecución:**
  - apache2
  - php8.2-fpm
  - mariadb
  - autoquiray-queue
  - Contenedor Docker plausible (opcional, solo para analítica).
- **Verificación rápida:**

```
sudo systemctl status apache2 php8.2-fpm mariadb autoquiray-queue
```

```
docker ps --filter "name=plausible"
```

- **Visualización:** los usuarios del grupo **Alumnado** tienen solo permisos de navegación. Cualquier modificación de ficheros en el servidor está restringida.

### 8.3 Acceso a recursos (Active Directory)

- **Profesores:** validan con credenciales del dominio para acceder a carpetas personal y colaborativa.
- **Dirección:** privilegios de auditoría sobre las carpetas del profesorado.
- **Nuevos usuarios:** se crean a partir de **objetos plantilla** para estandarizar pertenencias a grupos y GPOs.

### 8.4 Acceso de desarrollador (modo XAMPP local)

Para entornos de desarrollo en Windows sin Apache nativo, también se admite XAMPP:

1. Copiar el proyecto en C:\xampp\htdocs\autoquiray.
2. Iniciar **Apache** y **MySQL** desde el panel XAMPP.
3. Configurar .env con la base de datos local.
4. Ejecutar:

```
composer install
```

```
php artisan key:generate
```

php artisan migrate --seed

npm install && npm run build

5. Acceder a <http://localhost/autoquiray/public>.

## 9. Conclusión de la fase

La fase de implementación cumple el PMV definido en planificación:

- Red segmentada en VLANs operativa (real + simulada en Packet Tracer).
- Servidor Apache + MariaDB sirviendo AUTOQUIRAY en el aula.
- AD/DHCP funcionando contra los terminales.
- Plausible recogiendo eventos de uso desde el frontend, aislado en Docker.
- Repositorio Git en GitHub con histórico completo.

Quedan como mejoras (fase siguiente): HTTPS con Let's Encrypt, migración completa del frontend a Vite + Tailwind y automatización del despliegue con un script o GitHub Actions.

## Conclusión

### *Balance del proyecto*

El proyecto AUTOQUIRAY se planteó con un doble objetivo: por un lado, diseñar y desplegar la infraestructura de red completa de una autoescuela (segmentación por VLANs, AD, DHCP, NAS, backups) y, por otro, desarrollar la plataforma web que sostiene la operativa académica diaria (tests teóricos, reserva de clases prácticas, dashboards de profesorado y administración, consultas alumno–profesor).

Tras la fase de implementación, el balance es positivo:

- **Red lógica completa.** El diseño de VLANs (10/20/30/40/50/60 + segmentos de servidor) se ha validado tanto físicamente sobre el switch de capa 3 del aula como simulado en Cisco Packet Tracer (.pkt). Las restricciones por rol funcionan: el alumnado solo accede al servidor web; profesorado y dirección acceden a los recursos compartidos del dominio.
- **Servidor real operativo.** Apache 2.4 + PHP-FPM 8.2 + MariaDB sirven AUTOQUIRAY sobre Ubuntu Server. AD y DHCP corren sobre Windows Server. El worker de cola de Laravel está controlado por systemd y el firewall UFW restringe los puertos expuestos.
- **Aplicación web funcional.** Los tres roles (student, teacher, administrator) tienen sus flujos completos: autenticación con verificación de rol, tests con corrección y resultados, reservas de clases con bloqueo de cancelación si la clase ha comenzado, bandeja de consultas con respuestas y dashboards agregados con SQL.
- **Telemetría aislada.** Plausible Analytics corre en Docker, independiente del servidor web. Permite ver eventos de uso sin acoplar la analítica al stack de producción.
- **Trazabilidad de cambios.** Todo el código vive en GitHub (Timmo230/autoquiray) con ramas separadas para desarrollo (web) y entrega (main).

## Cumplimiento de objetivos

|                                                 |                                                                                 |
|-------------------------------------------------|---------------------------------------------------------------------------------|
| Red estructurada con VLANs por departamento     | Cumplido (físico + Packet Tracer)                                               |
| Servidor central de dominio, web y BD           | Cumplido (Apache + MariaDB + AD + DHCP)                                         |
| Plataforma web para tests, reservas y consultas | Cumplido                                                                        |
| Políticas de seguridad, backup y accesos        | Parcial — UFW, GPOs y plan de backup definidos; copias automatizadas pendientes |
| Conectividad estable entre equipos e internet   | Cumplido                                                                        |

## Dificultades encontradas

- **Materiales no disponibles a tiempo.** Falta de switches adicionales, NAS Synology y de las 28 estaciones finales del plan original. Solución: virtualización de servidores y simulación en Cisco Packet Tracer.
- **Reorientación parcial a Windows Server.** El plan inicial planteaba todo sobre Ubuntu. Al priorizar usabilidad del personal y plazos de entrega, AD y DHCP se trasladaron a Windows Server; el servidor web se mantiene en Linux.
- **Pipeline de frontend mixto.** Vite y Tailwind 4 declarados pero la carga real de assets se hace manual (CSS/JS desde resources/ y Bootstrap desde public/node\_modules). Decisión consciente para no bloquear el desarrollo; queda como deuda técnica.
- **Tipos de clave primaria mixtos.** Algunas tablas usan IDs string/UUID y otras id() autoincremental. Forzó cuidado extra al escribir joins en EmployeesPlaceService.
- **Typo histórico** message. Heredado del esquema original, conservado para no romper migraciones ya ejecutadas.

## Mejoras y siguientes pasos

Líneas claras de evolución que quedan abiertas:

1. **HTTPS con Let's Encrypt.** Servir AUTOQUIRAY bajo TLS con renovación automática vía certbot.
2. **Despliegue automatizado.** Pipeline en GitHub Actions: lint con Pint, tests con PHPUnit y `git pull + composer install + php artisan migrate` en el servidor tras merge a main.
3. **Migración completa a Vite + Tailwind 4.** Eliminar la carga manual de assets y consolidar el pipeline moderno.
4. **Backups automatizados al NAS.** Script de `mysqldump` cifrado y rotación de copias incrementales/semanales/mensuales según el plan definido.
5. **Monitorización adicional.** Complementar Plausible con un Prometheus + Grafana para métricas de sistema (CPU, RAM, latencia del worker, conexiones a MariaDB).
6. **Tests de integración.** Ampliar la suite de PHPUnit más allá de las pruebas funcionales actuales.
7. **Modo quiosco real en aulas.** Bloquear el navegador en los miniordenadores de alumnado solo a la URL de tests.

## ***Reflexión final***

AUTOQUIRAY no es solo una página web. Es la red, los servidores, las políticas de acceso, la base de datos, el pipeline de despliegue y la analítica de uso, todo orquestado para sustituir un proceso manual de autoescuela por uno centralizado. La realización del proyecto ha exigido aplicar competencias transversales del ciclo SMR: redes (VLAN, trunk, DHCP), administración de sistemas (Linux y Windows Server, AD, GPO), administración de bases de datos (MariaDB, esquema relacional), desarrollo web (Laravel, Blade, SCSS, JS, Bootstrap), control de versiones (Git/GitHub) y virtualización/contenedores (VirtualBox, Docker).

El resultado es un PMV defendible: la autoescuela puede operar sobre la plataforma desde el primer día y la arquitectura está preparada para escalar.

## **Bibliografía y webgrafía**

### ***Documentación oficial — Backend***

- **Laravel 12 — Documentación oficial.** Laravel LLC. <https://laravel.com/docs/12.x>
- **Laravel Eloquent ORM.** <https://laravel.com/docs/12.x/eloquent>
- **Laravel Blade Templates.** <https://laravel.com/docs/12.x/blade>
- **Laravel Queues.** <https://laravel.com/docs/12.x/queues>
- **Laravel Pint (linter PSR-12).** <https://laravel.com/docs/12.x/pint>
- **Laravel Pail (log viewer).** <https://github.com/laravel/pail>
- **PHP 8.2 — Manual.** The PHP Group. <https://www.php.net/manual/es/>
- **Composer — Dependency Manager for PHP.** <https://getcomposer.org/doc/>
- **PHPUnit.** Sebastian Bergmann. <https://phpunit.de/documentation.html>
- **Mockery.** <https://docs.mockery.io/>
- **FakerPHP.** <https://fakerphp.org/>

### ***Documentación oficial — Frontend***

- **Bootstrap 5.** <https://getbootstrap.com/docs/5.3/>
- **Sass / SCSS — Documentación.** <https://sass-lang.com/documentation/>
- **Tailwind CSS 4.** <https://tailwindcss.com/docs>
- **Vite.** Evan You. <https://vitejs.dev/guide/>
- **@tailwindcss/vite\*\* plugin.\*\*** <https://tailwindcss.com/docs/installation/using-vite>
- **Axios.** <https://axios-http.com/docs/intro>
- **MDN Web Docs.** Mozilla. <https://developer.mozilla.org/es/>

### ***Servidor web y base de datos***

- **Apache HTTP Server 2.4 — Documentation.** Apache Software Foundation. <https://httpd.apache.org/docs/2.4/>
- **mod\_rewrite.** [https://httpd.apache.org/docs/2.4/mod/mod\\_rewrite.html](https://httpd.apache.org/docs/2.4/mod/mod_rewrite.html)
- **mod\_proxy\_fcgi.** [https://httpd.apache.org/docs/2.4/mod/mod\\_proxy\\_fcgi.html](https://httpd.apache.org/docs/2.4/mod/mod_proxy_fcgi.html)

- **MariaDB Server — Knowledge Base.** MariaDB Foundation. <https://mariadb.com/kb/en/>
- **MySQL 8.0 — Reference Manual.** Oracle. <https://dev.mysql.com/doc/refman/8.0/en/>
- **phpMyAdmin.** <https://docs.phpmyadmin.net/>
- **PHP-FPM.** <https://www.php.net/manual/es/install.fpm.php>

## ***Contenedores y analítica***

- **Docker — Get Started.** Docker Inc. <https://docs.docker.com/get-started/>
- **Docker Compose.** <https://docs.docker.com/compose/>
- **Plausible Community Edition — Self-hosting.** Plausible Insights OÜ. <https://plausible.io/docs/self-hosting>
- **Plausible — Custom Events.** <https://plausible.io/docs/custom-event-goals>

## ***Redes y simulación***

- **Cisco Packet Tracer — Documentation.** Cisco Networking Academy. <https://www.netacad.com/courses/packet-tracer>
- **Cisco IOS — VLAN Configuration Guide.** Cisco Systems. [https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst2960/software/release/15-0\\_2\\_se/configuration/guide/scg2960/swvlan.html](https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst2960/software/release/15-0_2_se/configuration/guide/scg2960/swvlan.html)
- **RFC 2131 — Dynamic Host Configuration Protocol.** IETF. <https://datatracker.ietf.org/doc/html/rfc2131>
- **IEEE 802.1Q — VLAN tagging.** [https://standards.ieee.org/standard/802\\_1Q-2018.html](https://standards.ieee.org/standard/802_1Q-2018.html)

## ***Sistemas y administración***

- **Ubuntu Server — Documentation.** Canonical. <https://ubuntu.com/server/docs>
- **Windows Server — Active Directory Domain Services.** Microsoft Learn. <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/active-directory-domain-services>
- **Microsoft — DHCP Server role.** <https://learn.microsoft.com/en-us/windows-server/networking/technologies/dhcp/dhcp-top>
- **systemd\*\* — Manual.\*\*** freedesktop.org. <https://www.freedesktop.org/software/systemd/man/>
- **UFW — Uncomplicated Firewall.** Ubuntu Wiki. <https://help.ubuntu.com/community/UFW>
- **OpenSSH — Manual.** <https://www.openssh.com/manual.html>
- **VirtualBox — User Manual.** Oracle. <https://www.virtualbox.org/manual/>
- **Hyper-V — Documentation.** Microsoft Learn. <https://learn.microsoft.com/en-us/virtualization/hyper-v-on-windows/>
- **Synology DSM — Knowledge Center.** Synology Inc. <https://kb.synology.com/>

## **Control de versiones**

- **Git — Documentation.** Software Freedom Conservancy. <https://git-scm.com/doc>
- **Pro Git book.** Scott Chacon, Ben Straub. <https://git-scm.com/book/es/v2>
- **GitHub Docs.** GitHub Inc. <https://docs.github.com/es>
- **Conventional Commits 1.0.0.** <https://www.conventionalcommits.org/es/v1.0.0/>

## **Herramientas de desarrollo**

- **Visual Studio Code — Docs.** Microsoft. <https://code.visualstudio.com/docs>
- **PHP Intelephense (extensión VS Code).** <https://marketplace.visualstudio.com/items?itemName=bmewburn.vscode-intelephense-client>
- **Laravel Blade Snippets.** <https://marketplace.visualstudio.com/items?itemName=onecentlin.laravel-blade>
- **GitLens.** <https://marketplace.visualstudio.com/items?itemName=eamodio.gitlens>
- **DBeaver Community.** <https://dbeaver.io/docs/>
- **Postman Learning Center.** <https://learning.postman.com/>
- **curl — Manual.** <https://curl.se/docs/>

## **Estándares y buenas prácticas**

- **PSR-12 — Extended Coding Style.** PHP-FIG. <https://www.php-fig.org/psr/psr-12/>
- **OWASP Top 10 (2021).** Open Worldwide Application Security Project. <https://owasp.org/Top10/es/>
- **WCAG 2.2 — Web Content Accessibility Guidelines.** W3C. <https://www.w3.org/TR/WCAG22/>
- **Semantic Versioning 2.0.0.** Tom Preston-Werner. <https://semver.org/lang/es/>

## **Repositorio del proyecto**

- **AUTOQUIRAY — repositorio GitHub.** Quiriquino M., Sampalo Y. <https://github.com/Timmo230/autoquiray>
  - Rama main — entrega.
  - Rama web — desarrollo activo.

## **Material del centro**

- González Luna, J. L. — *Guía de la Fase de Planificación del Proyecto Intermodular*, 2º SMR. IES Saladillo, Algeciras (2025/2026).
- Departamento de Informática — *Comunicado de entrega de proyectos. Primera Convocatoria Ordinaria Junio 2026*. IES Saladillo (mayo 2026).
- Departamento de Informática — *Rúbrica de evaluación del proyecto intermodular*. IES Saladillo (mayo 2026).